

AWS Self-Learning and Hands-On Implementation

Topics

1. AWS Web Application Firewall (AWS WAF)
2. AWS Systems Manager Parameter Store

AWS Web Application Firewall (AWS WAF)

1. Introduction to AWS WAF

AWS WAF (Web Application Firewall) is a managed AWS security service used to monitor and control **HTTP and HTTPS requests** that are sent to web applications.

A web application is exposed to the internet and can receive thousands or millions of requests. Not every request is legitimate. Some requests may be malicious, automated, excessive, or designed to exploit vulnerabilities in the application.

AWS WAF allows us to define security rules that inspect incoming web requests and decide whether the request should be:

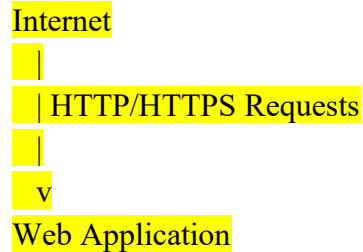
- Allowed
- Blocked
- Counted for monitoring
- Challenged, depending on the rule and configuration

The basic purpose can be summarized as:

AWS WAF acts as a security layer between incoming web traffic and the protected application, inspecting requests and applying configured rules before the requests reach the application.

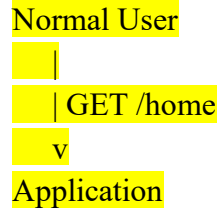
2. Why AWS WAF is Required

Consider a web application:



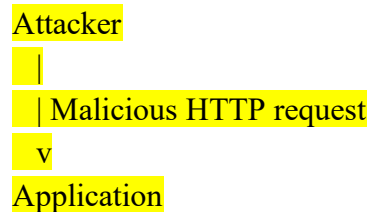
Anyone on the internet can potentially send requests to the application.

For example:



This is legitimate traffic.

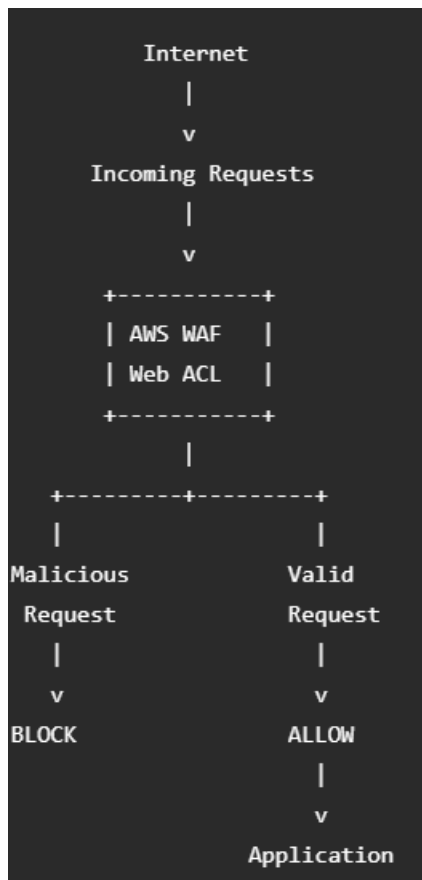
But an attacker might send:



The application may then be exposed to attacks such as:

- SQL injection
- Cross-site scripting
- Malicious bots
- Excessive requests
- IP-based attacks
- Suspicious request patterns
- Application-layer attacks

AWS WAF provides a controlled inspection layer:



3. What Does AWS WAF Protect?

AWS WAF is designed primarily for **web/application-layer traffic**, especially HTTP and HTTPS traffic.

It can protect supported AWS resources such as:

- Amazon CloudFront
- Application Load Balancer
- Amazon API Gateway
- AWS AppSync
- Amazon Cognito user pools
- AWS Verified Access
- Other supported AWS application resources

The exact resources available depend on the WAF scope and AWS service integration.

4. AWS WAF vs Traditional Firewall

It is important to understand this distinction.

A traditional network firewall generally operates at the **network/transport level**, controlling things such as:

IP address

Port

Protocol

For example:

Allow TCP 443

Block TCP 23

AWS WAF focuses on **web requests**.

It can inspect things such as:

IP address

HTTP method

URI path

Query string

Headers

Cookies

Request body

Request rate

Geographic origin

Example:

GET /login?user=admin

Host: example.com

User-Agent: browser

Source IP: 203.0.113.10

WAF can evaluate these characteristics against configured rules.

5. AWS WAF Important Components

Understanding the following components is very important.

5.1 Web ACL

Web ACL stands for **Web Access Control List**.

It is the primary configuration object in AWS WAF.

Think of a Web ACL as a **security rulebook**.

Example:

Web ACL

|

+-- Rule 1 → Block malicious IP

|

+-- Rule 2 → AWS Managed Rules

|

+-- Rule 3 → Rate limiting

|

+-- Rule 4 → Block specific request pattern

|

+-- Default Action → Allow

The Web ACL contains rules that determine how requests should be handled.

6. Rules

A **rule** defines a condition that AWS WAF evaluates against incoming requests.

For example:

IF

source IP belongs to blocked IP set

THEN

BLOCK

Another example:

IF
request rate exceeds configured limit

THEN
BLOCK

Another:

IF
request contains suspicious SQL injection pattern

THEN
BLOCK

Rules are therefore the actual logic used to inspect traffic.

7. Rule Statements

A rule statement defines **what AWS WAF should look for**.

Examples include:

IP address match

IF source IP = blocked IP

Geographic match

IF request originates from selected country

String match

IF URI contains "admin"

SQL injection match

IF request matches SQL injection patterns

Cross-site scripting match

IF request matches XSS patterns

Rate-based statement

IF request count exceeds configured threshold

8. Rule Actions

Once a rule matches a request, AWS WAF can perform an action.

Common actions include:

Allow

The request is allowed to continue.

Request

|

v

WAF

|

v

ALLOW

|

v

Application

Block

The request is rejected.

Request

|

v

WAF

|

v

BLOCK

|
X
Application

Count

The request is allowed, but AWS WAF records the match.

This is particularly useful when testing a new rule.

Instead of immediately blocking legitimate users, we can first use:

COUNT

and observe the results.

CAPTCHA / Challenge

AWS WAF also provides additional mechanisms for verifying whether traffic is likely coming from a legitimate user rather than automated traffic, depending on the configured rule and AWS WAF capabilities.

9. Default Action

A Web ACL also has a **default action**.

This is important because not every request necessarily matches a specific rule.

For example:

Rule 1 → Block malicious IP

Rule 2 → Block SQL injection

Rule 3 → Rate limit excessive requests

Default → ALLOW

If a request doesn't match Rules 1, 2, or 3, the default action is applied.

Another configuration could be:

Rule 1 → Allow trusted traffic

Default → BLOCK

This is useful when building a more restrictive access model.

10. AWS Managed Rules

Creating every security rule manually would be difficult.

AWS therefore provides **AWS Managed Rules**.

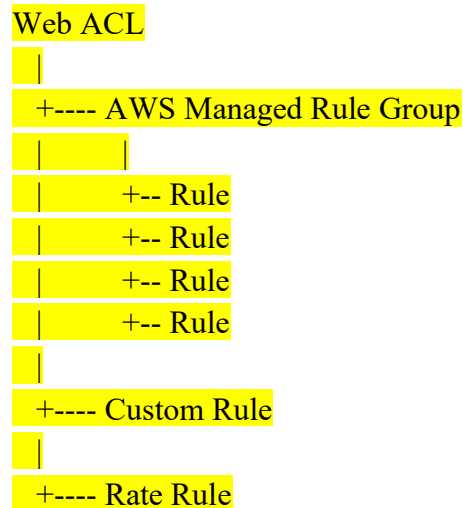
These are predefined rule groups maintained by AWS and/or AWS Marketplace security vendors.

One commonly used AWS-managed group is:

AWSManagedRulesCommonRuleSet

It provides protections for common web application attack patterns.

Architecture:



Managed rules are useful because security teams don't have to build every common detection pattern from scratch.

11. Custom Rules

AWS WAF also allows organizations to create their own rules.

Example:

Suppose an organization wants to block requests containing a specific value.

Request URI:

/admin/test

A custom rule could evaluate:

URI contains "/admin/test"

and then:

BLOCK

Custom rules are useful when the application's security requirements are specific to the organization.

12. Rate-Based Rules

A **rate-based rule** is used to control excessive request rates.

Imagine a website normally receives:

100 requests/minute

But suddenly one source sends:

10,000 requests/minute

This may indicate abusive or automated traffic.

A rate-based rule can identify excessive request rates and apply an action such as blocking or counting the requests.

Conceptually:

Incoming Requests

|

v

AWS WAF

|

v

Count requests

|

v

Threshold exceeded?

/\

YES NO

| |

BLOCK ALLOW

This is particularly useful for helping protect applications from excessive application-layer traffic.

13. IP Sets

AWS WAF allows you to create **IP sets**.

An IP set contains IP addresses or address ranges that you want to use in WAF rules.

Example:

Blocked IP Set

203.0.113.10

203.0.113.20

203.0.113.30

Then a WAF rule can say:

IF source IP belongs to Blocked IP Set

THEN BLOCK

This provides centralized management of IP addresses.

14. Regex Pattern Sets

WAF can also use pattern matching.

A **regex pattern set** allows multiple regular-expression patterns to be maintained and used in WAF rules.

Conceptually:

Regex Pattern Set

|

+-- Pattern 1

+-- Pattern 2

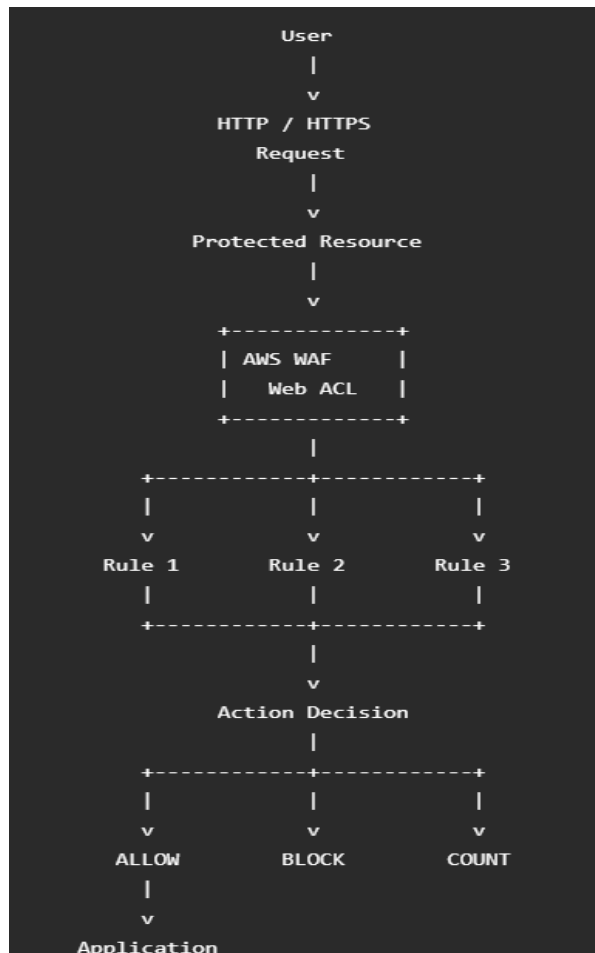
+-- Pattern 3

A rule can then inspect incoming requests against these patterns.

15. WAF Request Evaluation Flow

A simplified request flow is:

This is one of the most important flows to understand for your mentor discussion.



16. AWS WAF Logging and Monitoring

AWS WAF can be integrated with monitoring and logging services.

For example, WAF activity can be analyzed to understand:

- Which requests were blocked
- Which rules matched
- Source IP addresses
- Request patterns
- Rule evaluation results
- Allowed vs blocked traffic

This is important because simply creating a WAF rule isn't enough.

A DevOps/security engineer should also determine:

"Is my rule actually matching the traffic I expect, and am I accidentally blocking legitimate users?"

This is why **Count mode** can be useful when initially testing a rule.

17. AWS WAF and CloudWatch

AWS WAF metrics can be monitored using **Amazon CloudWatch**.

For example:

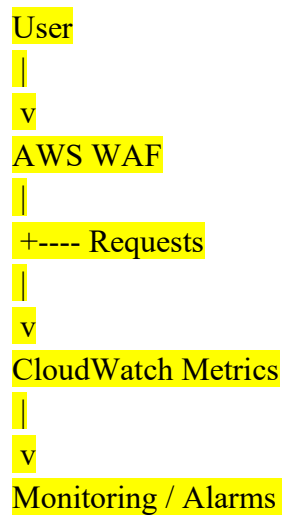
Allowed Requests

Blocked Requests

Counted Requests

This gives the operations team visibility into WAF activity.

A simplified monitoring architecture:



18. AWS WAF and AWS Shield

This is another important interview concept.

AWS WAF

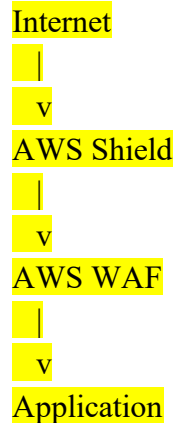
Primarily provides **web application-layer filtering and request inspection**.

AWS Shield

Provides protection against **DDoS attacks**, particularly at the network and transport layers, with different capabilities depending on the Shield offering.

They can work together.

Conceptually:

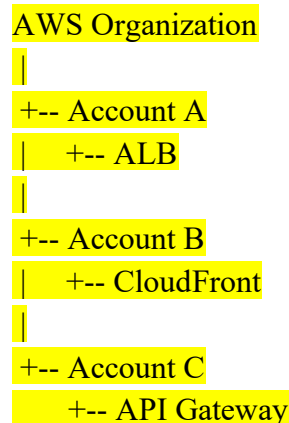


You should not describe WAF and Shield as the same service.

19. AWS WAF and Firewall Manager

AWS Firewall Manager is used for centralized security policy management across AWS accounts and resources in an AWS Organizations environment.

For example, a large organization might have:



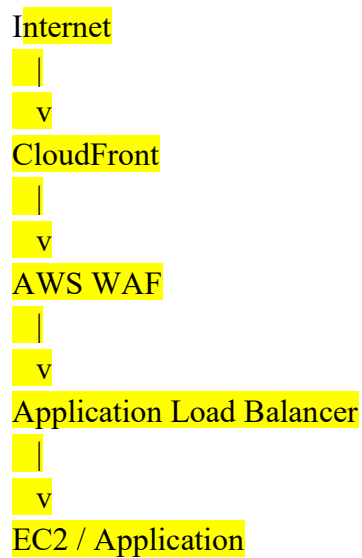
Firewall Manager can help centrally manage firewall/security policies across the environment.

20. WAF Real-World Example

Suppose we have:

<https://example.com>

The architecture is:

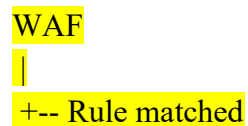


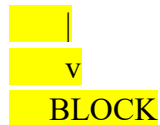
Suppose an attacker sends thousands of requests.

WAF can evaluate:

- Source IP
- Request URI
- HTTP headers
- Query parameters
- Request rate
- Request patterns

If the request matches a blocking rule:





The request does not continue to the application.

21. Benefits of AWS WAF

Major benefits include:

- Managed AWS security service
- No need to maintain firewall servers
- Customizable rules
- AWS Managed Rules
- Rate-based protection
- IP filtering
- HTTP/HTTPS request inspection
- Integration with AWS services
- Monitoring and logging capabilities
- Centralized Web ACL configuration

22. Limitations / Considerations

WAF should not be considered a complete replacement for all security controls.

For example:



These controls serve different purposes.

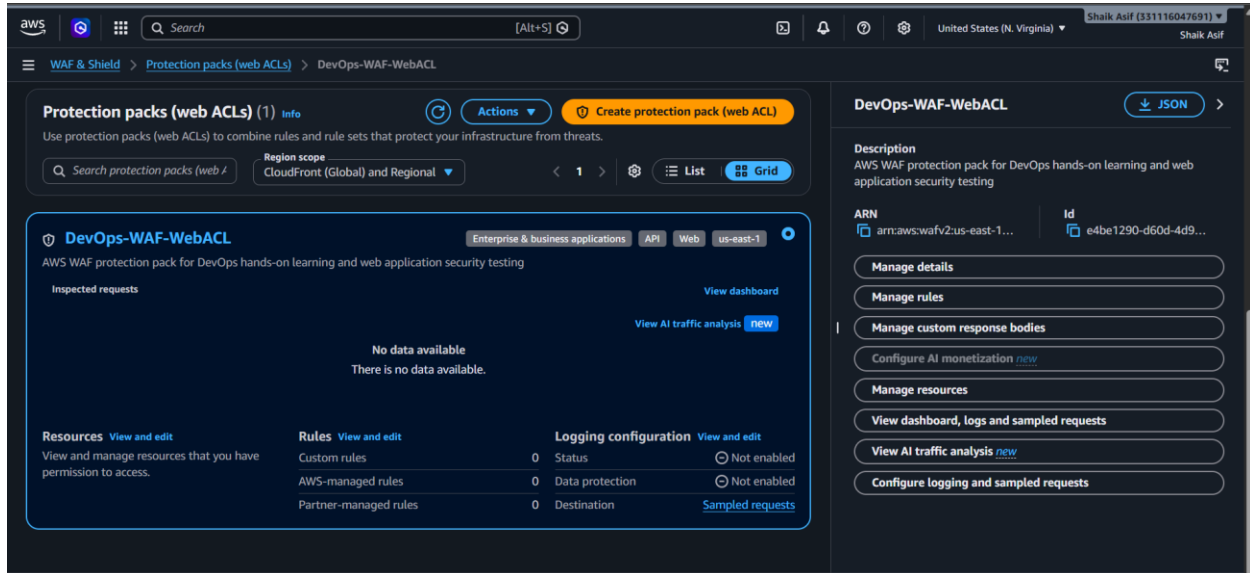
A secure AWS architecture generally uses **multiple layers of security**, rather than relying on a single service.

Task 1 — Create an AWS WAF Web ACL

Steps Used

1. Logged in to the **AWS Management Console**.
2. Selected the **US East (N. Virginia) – us-east-1** region.
3. Opened **AWS WAF & Shield**.
4. Navigated to **Protection packs (web ACLs)**.
5. Clicked **Create protection pack (web ACL)**.
6. Under **Tell us about your app**, selected:
 - a. **App category**: Enterprise & business applications
7. Under **Choose protection pack (web ACL) resource types**, selected:
 - a. **Application Load Balancer**
8. Skipped **Select resources to protect**, since no ALB was required for this initial creation task.
9. Under **Choose initial protections**, selected:
 - a. **Essentials**
10. Did not select the higher-cost **Recommended** protection package.
11. Under **Name and describe**, entered:
 - a. **Name**: DevOps-WAF-WebACL
 - b. **Description**: AWS WAF protection pack for DevOps hands-on learning and web application security testing
12. Opened **Customize protection pack (web ACL)**.
13. Reviewed the **Add rules** options but did not add additional custom, AWS Marketplace, or custom rule-group rules.
14. Reviewed the **Logging destination** configuration.
15. Did not configure CloudWatch logging for this initial task.
16. Created the protection pack by clicking **Create protection pack (web ACL)**.
17. Verified that DevOps-WAF-WebACL was successfully created.
18. Confirmed that the protection pack currently has:
 - a. **Custom rules**: 0
 - b. **AWS-managed rules**: 0
 - c. **Partner-managed rules**: 0
 - d. **Logging**: Not enabled

e. Associated resources: 0



Observation

The DevOps-WAF-WebACL protection pack was successfully created in the us-east-1 region for an **Application Load Balancer**. No resources are currently associated with the Web ACL, and no additional rules or logging have been configured. The dashboard shows no inspected-request data because no application traffic is currently passing through the Web ACL.

Conclusion

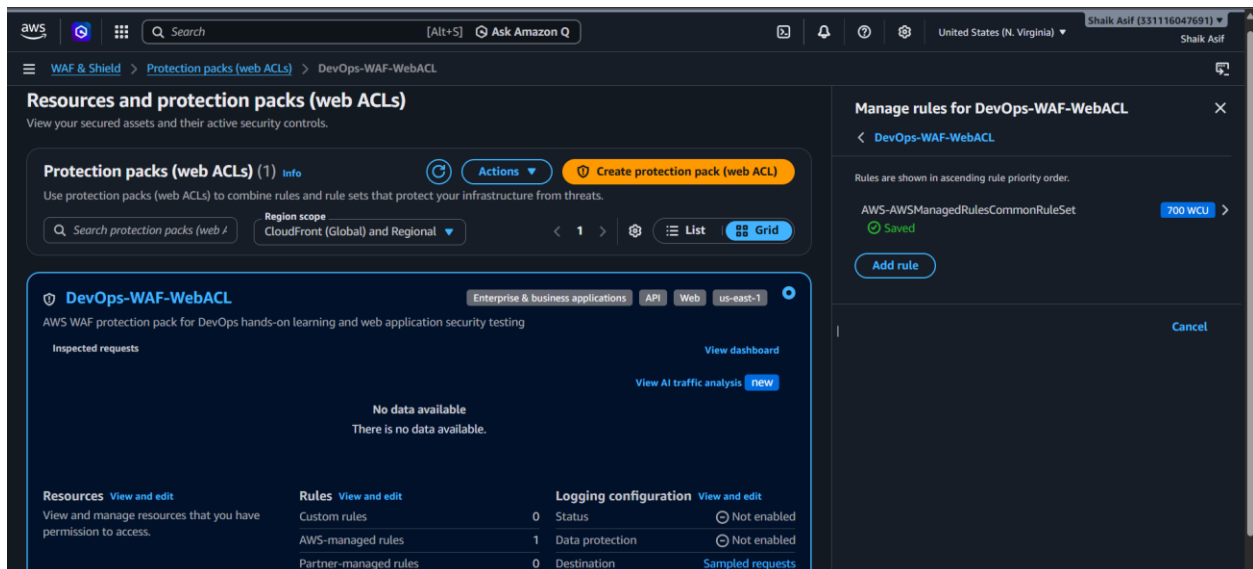
The AWS WAF Web ACL creation was completed successfully. The protection pack is now available for further configuration with **AWS-managed rules**, **custom rules**, **rate-based rules**, and **resource associations**. This task established the basic WAF configuration required for protecting web applications from common web-based threats.

Task 2 — Add an AWS-Managed Rule Group

Steps Used

1. Opened **AWS WAF & Shield** from the AWS Management Console.
2. Navigated to **Protection packs (web ACLs)**.
3. Selected the existing Web ACL:
 - a. DevOps-WAF-WebACL

4. Clicked **Manage rules**.
5. Clicked **Add rule**.
6. Selected **AWS-managed rule group**.
7. From the **Free** rule groups, selected:
 - a. **Core rule set**
8. Reviewed the AWS-managed rule group details:
 - a. Rule group: AWS-AWSManagedRulesCommonRuleSet
 - b. Version: **Default (Version 1.23)**
 - c. Capacity: **700 WCU**
 - d. Inspection: **All requests**
9. Reviewed the individual rules included in the Core Rule Set.
10. Kept the default rule actions as **Block**.
11. Did not select **Override rule group**.
12. Did not select **Override all rule actions**.
13. Saved the AWS-managed rule group.
14. Verified that the rule group appeared under **Manage rules**.
15. Confirmed:
 - a. **AWS-managed rules: 1**
 - b. **Custom rules: 0**
 - c. **Partner-managed rules: 0**
 - d. **Core Rule Set: Saved**



Observation

The AWS-managed **Core Rule Set** was successfully added to DevOps-WAF-WebACL. The rule group contains AWS-maintained security rules that inspect web requests for common web application threats.

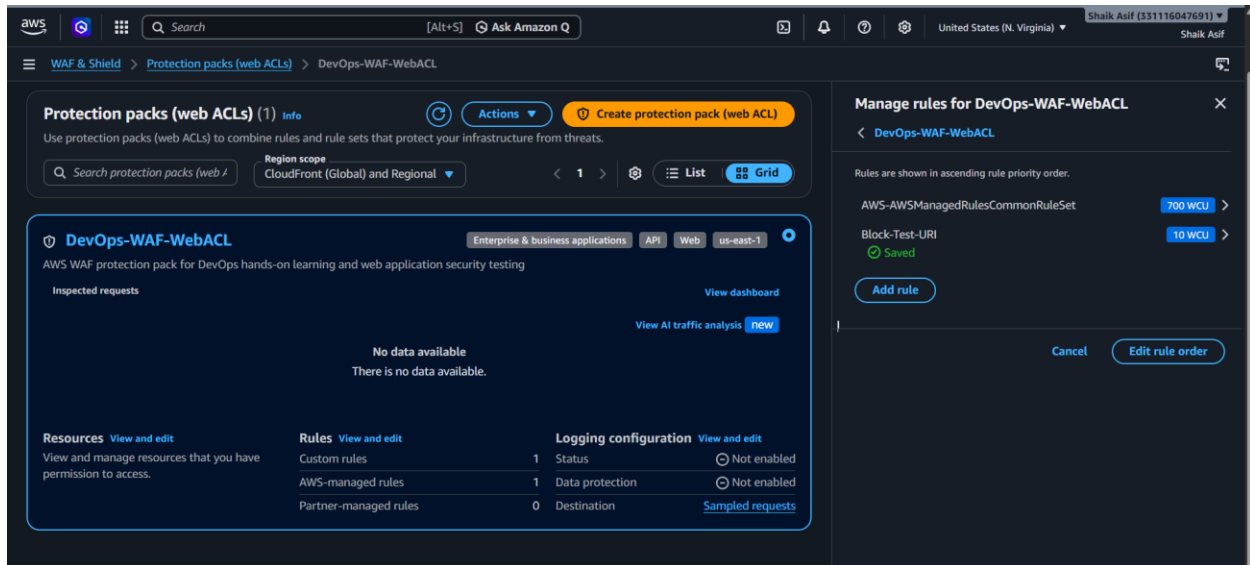
Conclusion

The Web ACL was successfully configured with an AWS-managed rule group. This demonstrates how AWS WAF can use preconfigured security rules maintained by AWS to provide protection against common web application vulnerabilities without manually creating every security rule.

Task 3 — Create a Custom WAF Rule

Steps Used

- Opened **AWS WAF & Shield** → **Protection packs (Web ACLs)**.
- Selected the Web ACL **DevOps-WAF-WebACL**.
- Opened **Manage rules** → **Add rule** → **Custom rule**.
- Selected **Visual** rule configuration.
- Provided the rule name as **Block-Test-URI**.
- Configured the rule to inspect the **URI path**.
- Set the match condition to **Contains string**.
- Entered **blocked-test** as the string to match.
- Configured the rule action as **Block**.
- Left **Custom response** disabled because it was not required for this task.
- Left **Labels** disabled because they were not required.
- Reviewed the configuration and added the custom rule to the Web ACL.
- Verified that **Block-Test-URI** was listed under the Web ACL rules.



Observation

The custom WAF rule was created successfully. The rule is configured to inspect incoming requests and identify requests whose **URI path contains blocked-test**. When a matching request reaches the associated resource, AWS WAF will apply the **Block** action.

Conclusion

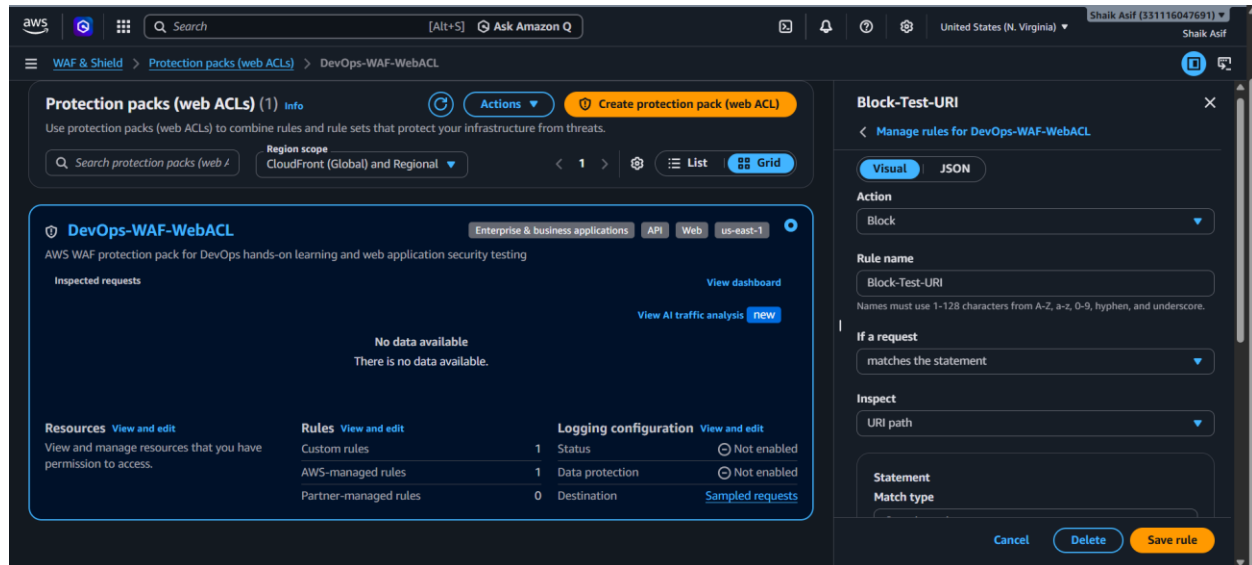
The custom AWS WAF rule was successfully configured to provide application-specific protection based on the URI path. This task demonstrated how custom WAF rules can be created using conditions and actions, allowing administrators to define their own request-filtering logic.

Task 4 — Test and Verify the WAF Rule

Steps Used

- Opened **AWS WAF & Shield** → **Protection packs (Web ACLs)**.
- Selected the Web ACL **DevOps-WAF-WebACL**.
- Opened **Manage rules**.
- Verified the custom rule **Block-Test-URI** was present.
- Verified that the rule inspects the **URI path**.
- Verified the match condition as **Contains string** with the value **blocked-test**.

- Verified that the rule action was configured as **Block**.
- Checked the **Associated AWS resources** section.
- Confirmed that the Web ACL currently had **0 associated resources**, so an actual application request test was not performed.



Observation

The Block-Test-URI custom rule was successfully configured and verified. It is designed to block requests when the URI path contains blocked-test. Since no resource was associated with the Web ACL, the rule could only be verified through its configuration and could not be tested with live application traffic.

Conclusion

The WAF custom rule was successfully created and verified. This task demonstrated how AWS WAF evaluates incoming requests based on URI-path conditions and applies the configured **Block** action when a request matches the rule.

AWS Systems Manager Parameter Store

23. Introduction

AWS Systems Manager Parameter Store is a capability of AWS Systems Manager that provides secure, hierarchical storage for **configuration data and secrets**.

Applications frequently require configuration values such as:

- Database endpoint
- Environment name
- Application URL
- Port number
- Feature flags
- API configuration
- Passwords
- Tokens

Instead of hardcoding these values into application code or configuration files, they can be stored centrally in Parameter Store.

24. Problem Without Parameter Store

Imagine an application contains:

```
database_host = "prod-db.example.com"  
database_password = "MyPassword123"
```

This creates several problems.

Problem 1 — Hardcoding

Configuration is embedded in application code.

Problem 2 — Security

Sensitive information may accidentally be committed to Git.

Problem 3 — Configuration changes

If the database changes, the application code may need to be changed.

Problem 4 — Multiple environments

You might have:

Development

Testing

Production

Each environment may require different values.

25. Parameter Store Solution

Instead:

Parameter Store

/dev/app/database-host

/test/app/database-host

/prod/app/database-host

The application retrieves the appropriate parameter.

Architecture:

Application

|

| Get parameter

v

Systems Manager

|

v

Parameter Store

|

+-- /dev/app/database-host

+-- /test/app/database-host

+-- /prod/app/database-host

This separates:

Application code from application configuration.

That is one of the most important concepts to remember.

26. Parameter Name Hierarchy

Parameter Store supports hierarchical parameter names.

For example:

```
/myapp/dev/database/host  
/myapp/dev/database/port  
/myapp/dev/database/name
```

```
/myapp/prod/database/host  
/myapp/prod/database/port  
/myapp/prod/database/name
```

This creates logical organization.

Think of it like folders:

```
myapp  
|  
|-- dev  
|   |  
|   |-- database  
|       |-- host  
|       |-- port  
|  
|-- prod  
|   |  
|   |-- database  
|       |-- host  
|       |-- port
```

This becomes very useful when managing many parameters.

27. Parameter Types

Parameter Store supports three main parameter types.

27.1 String

Used for a single text value.

Example:

Name:

/devops/app/environment

Type:

String

Value:

development

Other examples:

PORT = 8080

ENVIRONMENT = production

APP_NAME = payment-service

28. StringList

StringList is used when multiple values need to be stored in a single parameter.

Example:

Name:

/devops/app/servers

Type:

StringList

Value:

server1,server2,server3

Conceptually:

Parameter

|

+-- server1

+-- server2

+-- server3

The application can retrieve the value and process the individual entries.

29. SecureString

SecureString is designed for sensitive configuration values.

Example:

Name:

/devops/app/database/password

Type:

SecureString

Value:

<encrypted-value>

The value is encrypted rather than stored as ordinary plaintext.

AWS KMS is used as part of the encryption mechanism.

Conceptually:

Application

|

v

Parameter Store

|

v

SecureString

|

v

AWS KMS

|
v

Encrypted Value

For our hands-on lab, we should use a **dummy test secret**, not a real password or production credential.

30. Parameter Store and AWS KMS

For SecureString parameters, encryption is handled using AWS KMS.

Conceptually:

User/Application

|
v

Parameter Store

|
v

SecureString

|
v

AWS KMS

|
v

Encrypted parameter

When an authorized application retrieves the value with decryption enabled, AWS uses the appropriate KMS permissions to return the plaintext value.

Therefore, IAM permissions become important.

31. IAM and Parameter Store

This is an important DevOps concept.

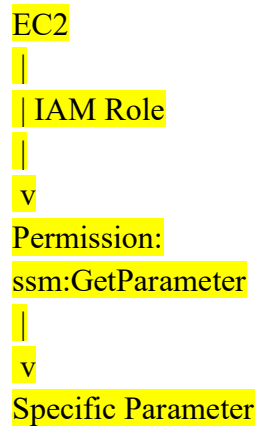
Suppose an EC2 application needs to retrieve:

/prod/database/password

The EC2 instance should not automatically have permission to read every parameter.

Instead, its IAM role should grant only the required permissions.

Conceptually:



For SecureString, additional KMS permissions may be required to decrypt the value.

This follows the **principle of least privilege**.

32. Example IAM Permission Concept

A permission might conceptually look like:

Action:

ssm:GetParameter

Resource:

/myapp/prod/database/password

Instead of giving:

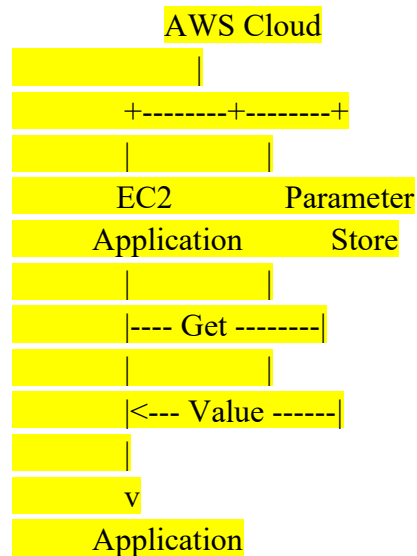
ssm:*

over all parameters.

This is a much better security practice.

33. Parameter Store with EC2

A common architecture is:

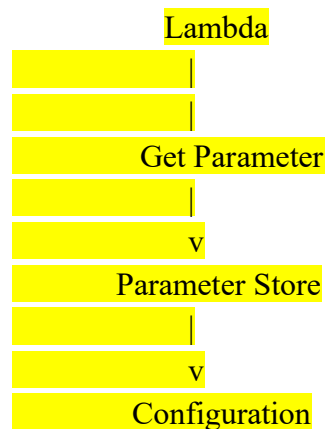


The EC2 application can retrieve configuration dynamically instead of storing credentials inside the server's application files.

34. Parameter Store with Lambda

Parameter Store is also commonly used with Lambda.

Example:



A Lambda function might retrieve:

/api/base-url

/database/host

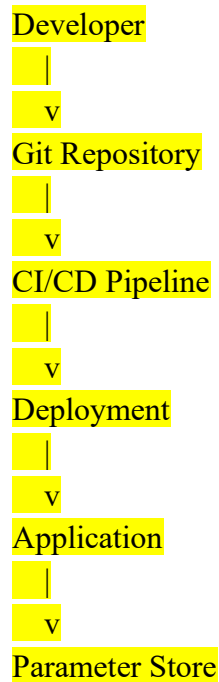
/application/environment

This helps keep configuration outside the Lambda source code.

35. Parameter Store with CI/CD

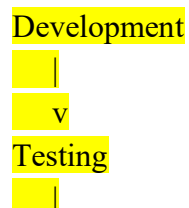
Parameter Store can also be useful in DevOps pipelines.

For example:



The pipeline/application can retrieve environment-specific configuration when deploying.

This supports:



without hardcoding environment-specific values into source code.

36. Standard vs Advanced Parameters

Parameter Store provides different parameter tiers.

Standard

Suitable for many basic configuration requirements.

Advanced

Provides additional capabilities such as:

- Larger parameter values
- Parameter policies
- Higher parameter limits
- Additional advanced functionality

For our learning lab, **Standard parameters are enough.**

This also helps keep the exercise simple and avoid unnecessary costs.

37. Parameter Policies

Advanced parameters can support parameter policies.

These can be useful for managing parameter lifecycle and expiration-related behavior.

For example, an organization might want a parameter to be reviewed or expire after a defined period.

This is more of an advanced production concept; we don't need it for the initial hands-on lab.

38. Parameter Store vs Secrets Manager

This is an important interview question.

Both services can store sensitive information, but they are designed with somewhat different use cases.

Feature	Parameter Store	Secrets Manager
Configuration storage	Yes	Yes
SecureString	Yes	Yes
KMS encryption	Yes	Yes
Hierarchical parameters	Yes	Different model
Automatic secret rotation	Limited/different approach	Strong built-in capability
Simple configuration values	Excellent fit	Possible
Purpose-built secret lifecycle	Less specialized	Stronger fit

Simple rule to remember:

Parameter Store is excellent for centralized application configuration and can also securely store sensitive values using SecureString. Secrets Manager is purpose-built for secret management and features such as managed rotation.

39. Parameter Store Security

Important security practices include:

1. Use SecureString for sensitive values

Don't store passwords as ordinary String parameters.

2. Use IAM least privilege

Give applications access only to parameters they actually need.

3. Use KMS appropriately

SecureString encryption relies on KMS.

4. Don't expose parameters unnecessarily

Avoid giving developers or applications access to unrelated production parameters.

5. Don't hardcode credentials

The purpose of centralized configuration is defeated if credentials remain inside source code.

40. Parameter Store Retrieval

Parameters can be retrieved using different mechanisms, including:

- AWS Management Console
- AWS CLI
- AWS SDKs
- AWS services/integrations

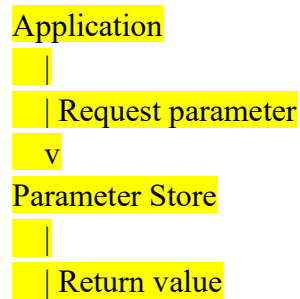
For example:

```
aws ssm get-parameter \  
  --name "/devops/app/environment" \  
  --region us-east-1
```

For a SecureString:

```
aws ssm get-parameter \  
  --name "/devops/app/test-secret" \  
  --with-decryption \  
  --region us-east-1
```

The important concept is:



v

Application

41. Real-World DevOps Example

Suppose we have an e-commerce application.

The application needs:

Application Environment

Database Host

Database Port

API Endpoint

Database Password

Instead of:

application.properties

DB_HOST=prod-db.example.com

DB_PASSWORD=abc123

we store:

/shop/prod/environment

/shop/prod/database/host

/shop/prod/database/port

/shop/prod/api/url

/shop/prod/database/password

Then:

Application

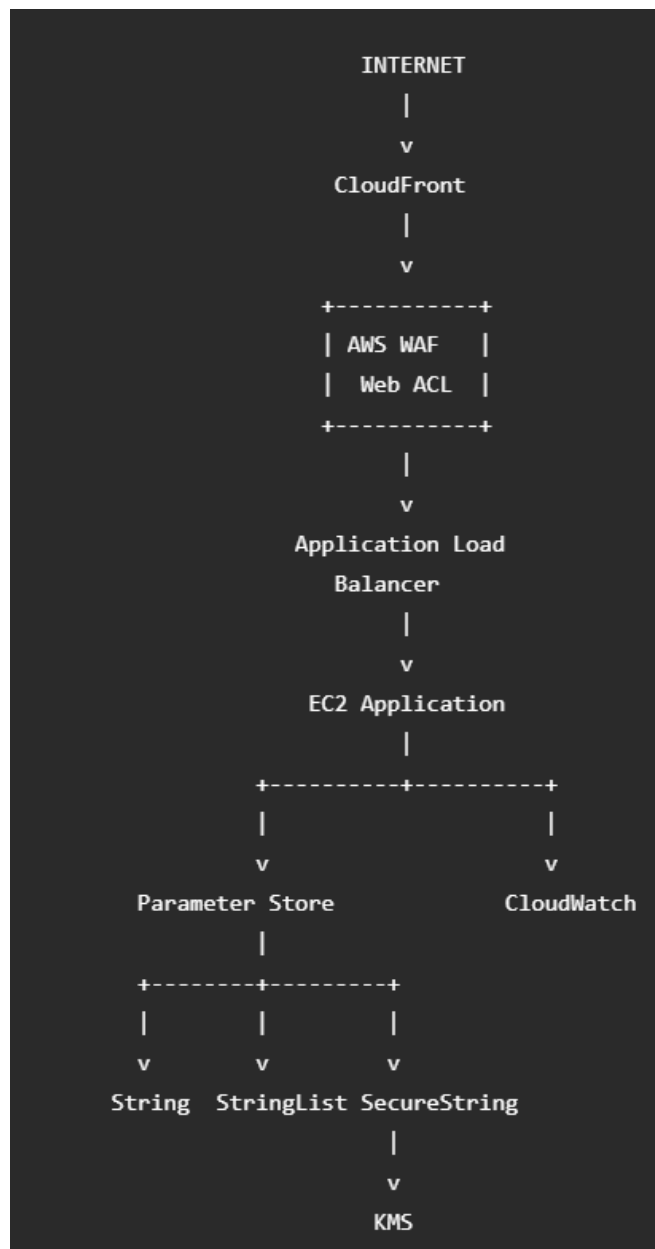
```
|
+-----> Parameter Store
|       |
|       +-- Environment
|       +-- DB Host
|       +-- DB Port
|       +-- API URL
|       +-- DB Password
```

|
v
Application starts

This provides centralized configuration management.

42. WAF + Parameter Store in a DevOps Architecture

We can combine both services into one application architecture:



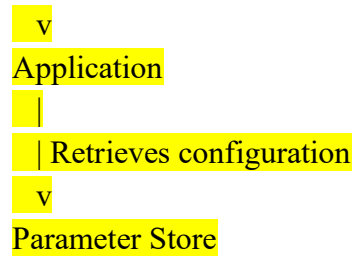
Security flow

Internet

|
v

WAF

|
| Filters malicious traffic



This is a good architecture to explain to your mentor because the two services solve **different problems**:

WAF

Protects incoming web traffic.

Parameter Store

Manages application configuration and sensitive parameter values.

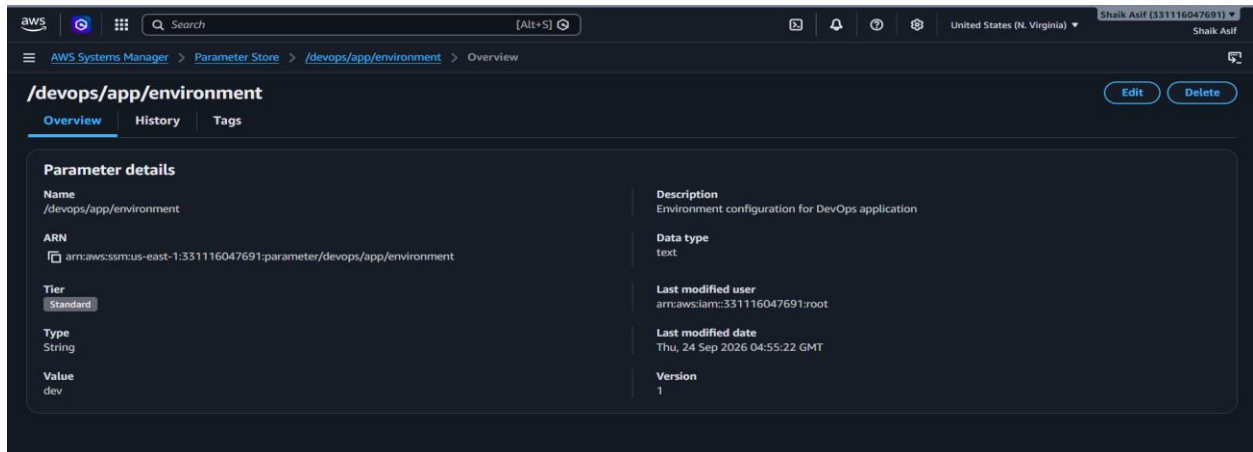
43. Key Differences to Remember

AWS WAF	Parameter Store
Security service	Configuration management service
Inspects web requests	Stores configuration/parameters
Protects web applications	Provides configuration to applications
Uses Web ACLs	Uses parameters
Uses rules	Uses parameter types
Can block requests	Can securely store values
Works with HTTP/HTTPS traffic	Works with application/service configuration
Supports managed/custom rules	Supports String/StringList/SecureString

Task 1 — Create a String Parameter in AWS Systems Manager Parameter Store

Steps Used

1. Opened **AWS Systems Manager** → **Parameter Store**.
2. Created a parameter: `/devops/app/environment`
3. Selected **Standard** tier.
4. Selected **String** as the parameter type.
5. Stored the value: `dev`
6. Verified the parameter using AWS CLI: `aws ssm describe-parameters --region us-east-1`



```
asifs@Asif_Shaik MINGW64 ~
$ aws ssm describe-parameters \
> --region us-east-1
{
  "Parameters": [
    {
      "Name": "/devops/app/environment",
      "ARN": "arn:aws:ssm:us-east-1:331116047691:parameter/devops/app/envi
ronment",
      "Type": "String",
      "LastModifiedDate": "2026-09-24T10:25:22.621000+05:30",
      "LastModifiedUser": "arn:aws:iam::331116047691:root",
      "Description": "Environment configuration for DevOps application",
      "Version": 1,
      "Tier": "Standard",
      "Policies": [],
      "DataType": "text"
    }
  ]
}
```

Observation

The parameter was successfully created in **US East (N. Virginia)**. The `describe-parameters` command confirmed that the parameter is of type **String**, has **Version 1**, and is using the **Standard** tier.

Conclusion

AWS Systems Manager Parameter Store was successfully used to create and manage a String parameter for storing non-sensitive application configuration data.

Task 2 — Create a SecureString Parameter

Objective

To create a **SecureString parameter** in AWS Systems Manager Parameter Store for securely storing sensitive application configuration data and verify access using the AWS CLI.

Steps Used

1. Opened the **AWS Management Console** and navigated to:
Systems Manager → Parameter Store.
2. Confirmed the AWS Region as:
US East (N. Virginia) — us-east-1.
3. Created a parameter with the following configuration:
 - a. **Name:** /devops/app/db-password
 - b. **Type:** SecureString
 - c. **Tier:** Standard
 - d. **Description:** Encrypted database password for DevOps application
 - e. **Test value:** DevOps@12345
4. Since AWS CLI was being used from **Git Bash**, enabled `MSYS_NO_PATHCONV=1` to prevent Git Bash from converting the Parameter Store path into a Windows filesystem path.
5. Used the following command to create the parameter:

```
MSYS_NO_PATHCONV=1 aws ssm put-parameter \  
--name "/devops/app/db-password" \  
--type "SecureString" \  
--value "DevOps@12345" \  

```

```
--description "Encrypted database password for DevOps application" \  
--region us-east-1
```

6. The CLI returned:

ParameterAlreadyExists

This confirmed that the parameter already existed, so it was not overwritten.

7. Verified the parameter using:

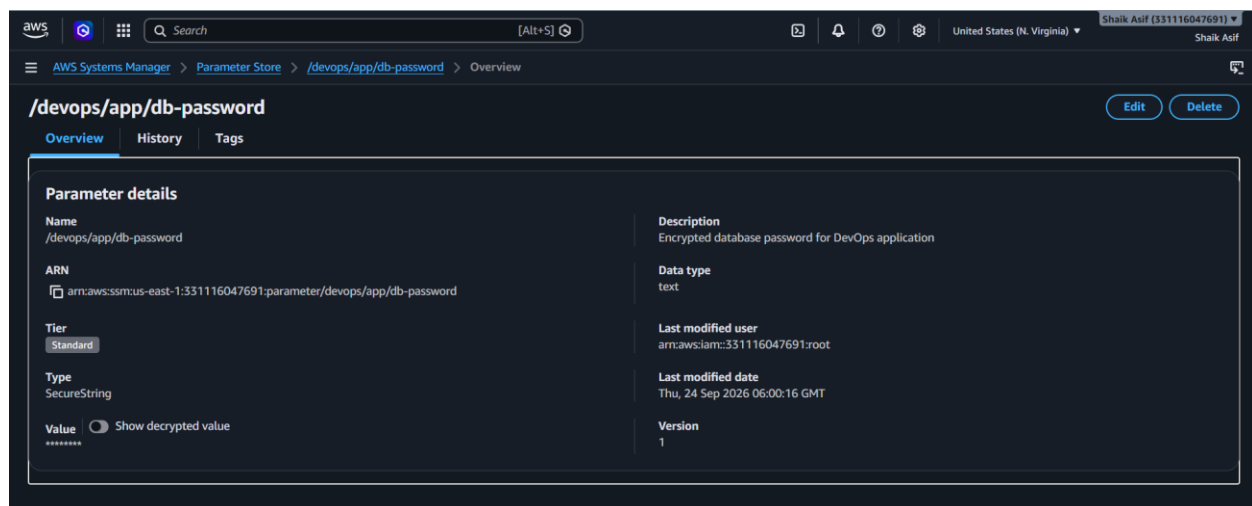
```
MSYS_NO_PATHCONV=1 aws ssm get-parameter \  
--name "/devops/app/db-password" \  
--region us-east-1
```

8. Used --with-decryption when required to retrieve the decrypted parameter value:

```
MSYS_NO_PATHCONV=1 aws ssm get-parameter \  
--name "/devops/app/db-password" \  
--with-decryption \  
--region us-east-1
```

9. Verified the parameter metadata using:

```
MSYS_NO_PATHCONV=1 aws ssm describe-parameters \  
--region us-east-1
```



```

MINGW64/C:/Users/asifs
asifs@Asif_Shaik MINGW64 ~
$ MSYS_NO_PATHCONV=1 aws ssm get-parameter \
> --name "/devops/app/db-password" \
> --region us-east-1
{
  "Parameter": {
    "Name": "/devops/app/db-password",
    "Type": "SecureString",
    "Value": "AQICAHF5mg1N9MO/YVMXV-w1Yf5NLWi+lm1VryjT+5jlgGAFxgkFknjh8C7OVXvdK134AAAAA3B08gkqhkiG9w0BwaguzBZAgEAWFQCSqSgIb3DQEHAeBgIghkgBZQMEAS4wEQQXDb+wj9NRORhTIC1ZAgEQGduqPdaeF8uJlKAYP5567xxUlyo
eKDCBwPTvpfpghDacB8ecJG0rw=",
    "Version": 1,
    "LastModifiedDate": "2026-09-24T11:30:16.365000+05:30",
    "ARN": "arn:aws:ssm:us-east-1:331116047691:parameter/devops/app/db-password",
    "DataType": "text"
  }
}

asifs@Asif_Shaik MINGW64 ~
$ MSYS_NO_PATHCONV=1 aws ssm get-parameter \
> --name "/devops/app/db-password" \
> --with-decryption \
> --region us-east-1
{
  "Parameter": {
    "Name": "/devops/app/db-password",
    "Type": "SecureString",
    "Value": "Devops812345",
    "Version": 1,
    "LastModifiedDate": "2026-09-24T11:30:16.365000+05:30",
    "ARN": "arn:aws:ssm:us-east-1:331116047691:parameter/devops/app/db-password",
    "DataType": "text"
  }
}

```

```

asifs@Asif_Shaik MINGW64 ~
$ MSYS_NO_PATHCONV=1 aws ssm describe-parameters \
> --region us-east-1
{
  "Parameters": [
    {
      "Name": "/devops/app/db-password",
      "ARN": "arn:aws:ssm:us-east-1:331116047691:parameter/devops/app/db-password",
      "Type": "SecureString",
      "KeyId": "alias/aws/ssm",
      "LastModifiedDate": "2026-09-24T11:30:16.365000+05:30",
      "LastModifiedUser": "arn:aws:iam::331116047691:root",
      "Description": "Encrypted database password for DevOps application",
      "Version": 1,
      "Tier": "Standard",
      "Policies": [],
      "DataType": "text"
    },
    {
      "Name": "/devops/app/environment",
      "ARN": "arn:aws:ssm:us-east-1:331116047691:parameter/devops/app/environment",
      "Type": "String",
      "LastModifiedDate": "2026-09-24T10:25:22.621000+05:30",
      "LastModifiedUser": "arn:aws:iam::331116047691:root",
      "Description": "Environment configuration for DevOps application",
      "Version": 1,
      "Tier": "Standard",
      "Policies": [],
      "DataType": "text"
    }
  ]
}

```

Observations

- The parameter was created as a **SecureString**.
- SecureString parameters are designed for sensitive configuration values.
- AWS Systems Manager Parameter Store integrates with **AWS KMS** for encryption.
- Access to decrypted values is controlled through IAM permissions.
- Git Bash can convert paths beginning with / into Windows paths. Using `MSYS_NO_PATHCONV=1` prevents this behavior when running AWS CLI commands.
- The ParameterAlreadyExists message confirmed that the parameter already existed and prevented accidental overwriting.

Conclusion

The SecureString parameter /devops/app/db-password was successfully created and verified in AWS Systems Manager Parameter Store. This task demonstrated how sensitive application configuration can be stored securely using **SecureString**, protected through **AWS KMS encryption**, and accessed through authorized AWS CLI operations.